Dynamic Property Ordering for Efficient Multi-Property Bounded Model Checking

Using Runtime Progress to Guide Verification Scheduling

Vivek Kumar

M.Tech, Computer Science

Under the guidance of
Prof. Ansuman Banerjee

Indian Statistical Institute Kolkata

February 18, 2026

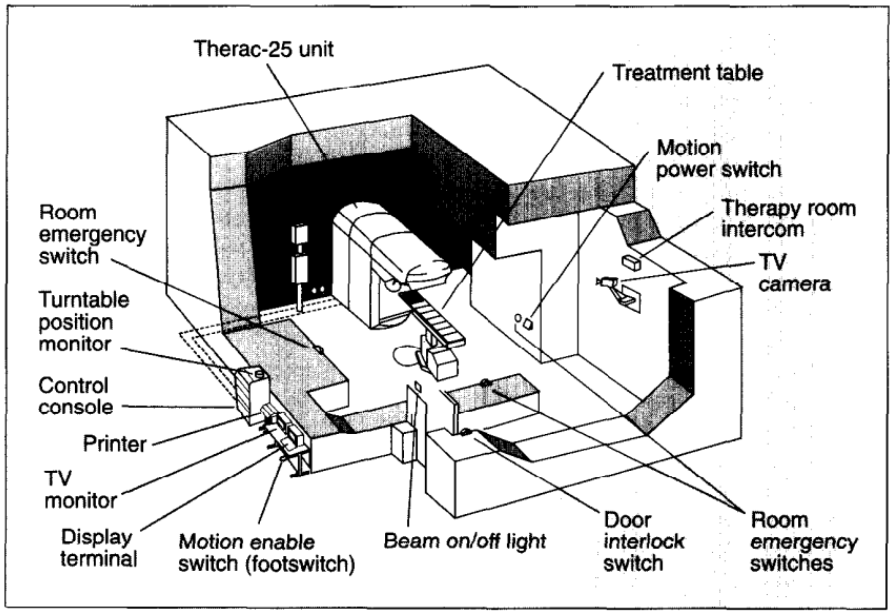
Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Outline

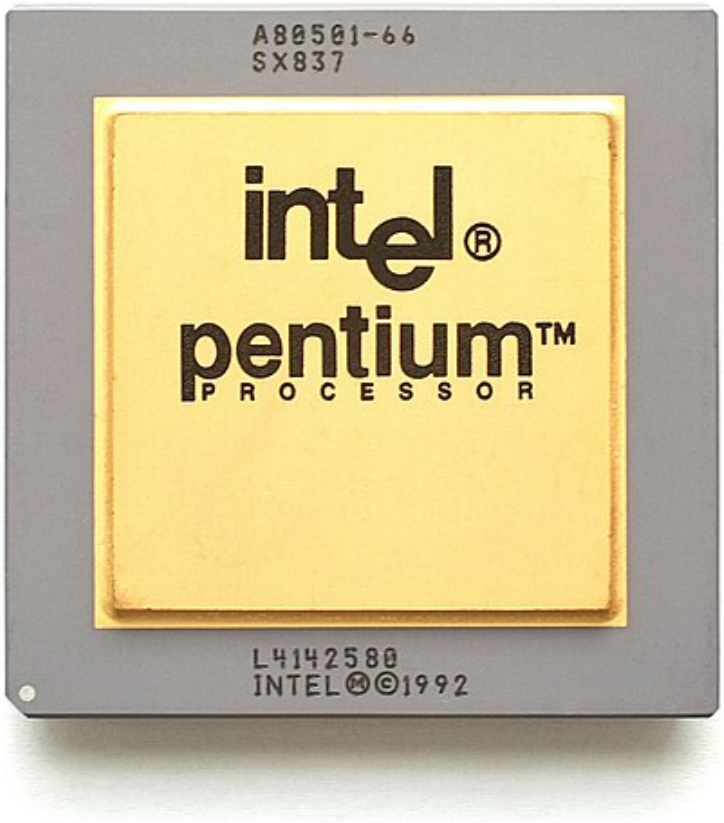
1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Therac-25 Radiation Therapy Accident (1985–1987)



- Computer-controlled radiation therapy machine
- At least 6 cases of overdoses ($\mathbf{100}\times$)
- 3 deaths
- Error in control software (race condition)

Intel Pentium FDIV Bug (1994)



- Intel Pentium floating-point division unit
- Certain floating point division operations performed produced incorrect results
- Byte magazine estimated that 1 in 9 billion floating point divides with random parameters would produce inaccurate results
- Caused by missing entries in lookup table
- Cost Intel **$475 million** recall + enormous image loss of Intel Corp.

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Verification Methodology

Simulation-Based Verification

- Input oriented (the designer supplies input tests)
- Possibility that an error escapes verification

Verification Methodology

Simulation-Based Verification

- Input oriented (the designer supplies input tests)
- Possibility that an error escapes verification

Verification Methodology

Simulation-Based Verification

- Input oriented (the designer supplies input tests)
- Possibility that an error escapes verification

Formal Verification

- Output oriented (the designer supplies the output properties)
- Exhaustively search all possible input and state conditions

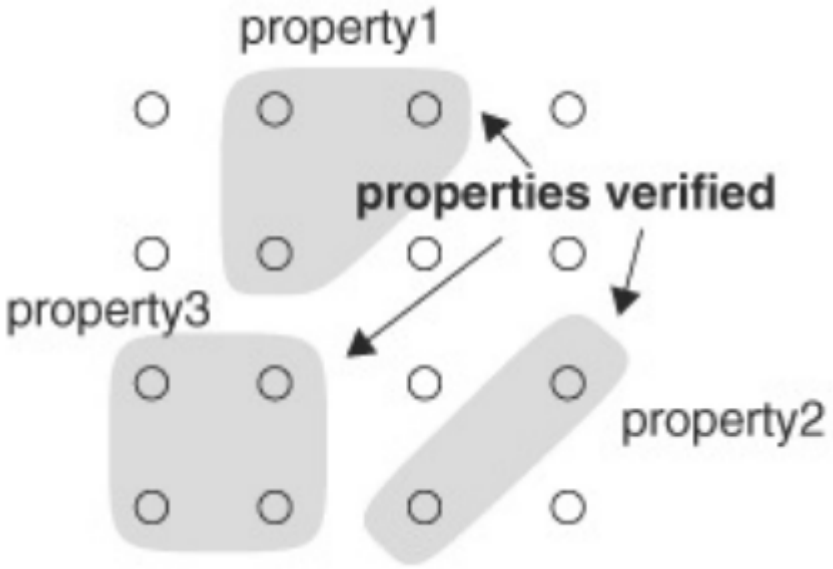
Verification Methodology

Simulation-Based Verification

- Input oriented (the designer supplies input tests)
- Possibility that an error escapes verification

Formal Verification

- Output oriented (the designer supplies the output properties)
- Exhaustively search all possible input and state conditions



Formal Verification

Definition

Formal verification is the process of mathematically proving that a system satisfies a given specification for **all possible behaviors**.

Input to Formal Verification

- System model (hardware / software design)
- Formal specification (properties)

Output

- Proof of correctness, or
- Counterexample (bug trace)

Techniques in Formal Verification

- Theorem Proving

Techniques in Formal Verification

- Theorem Proving
- Equivalence Checking

Techniques in Formal Verification

- Theorem Proving
- Equivalence Checking
- Abstract Interpretation

Techniques in Formal Verification

- Theorem Proving
- Equivalence Checking
- Abstract Interpretation
- **Model Checking**

Focus of this work

We focus on Model Checking

Example

Hardware Circuit

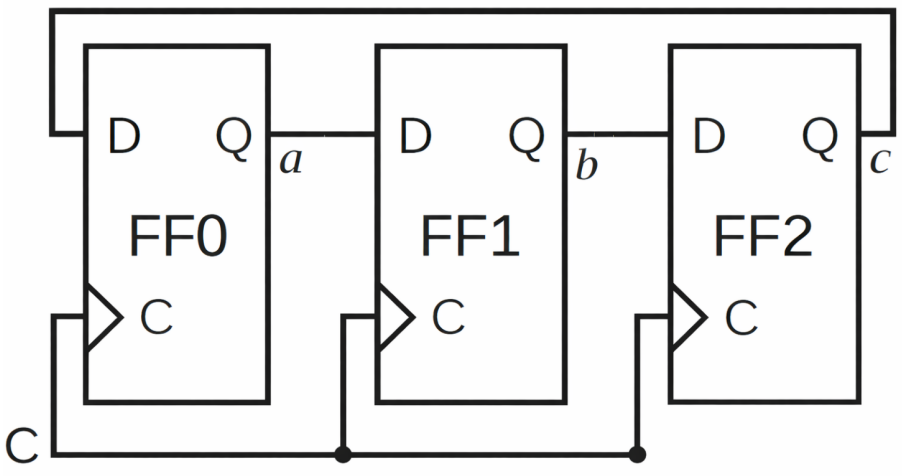


Figure: A 3-bit ring counter

State Transition System

Example

Hardware Circuit

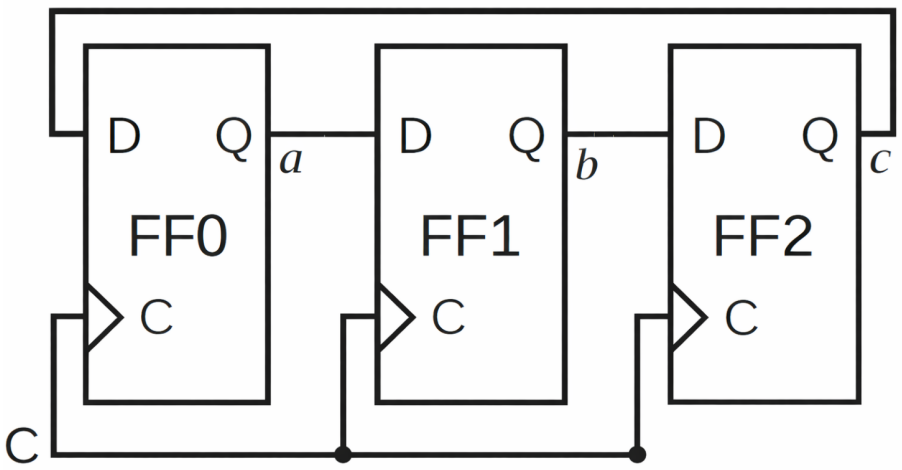
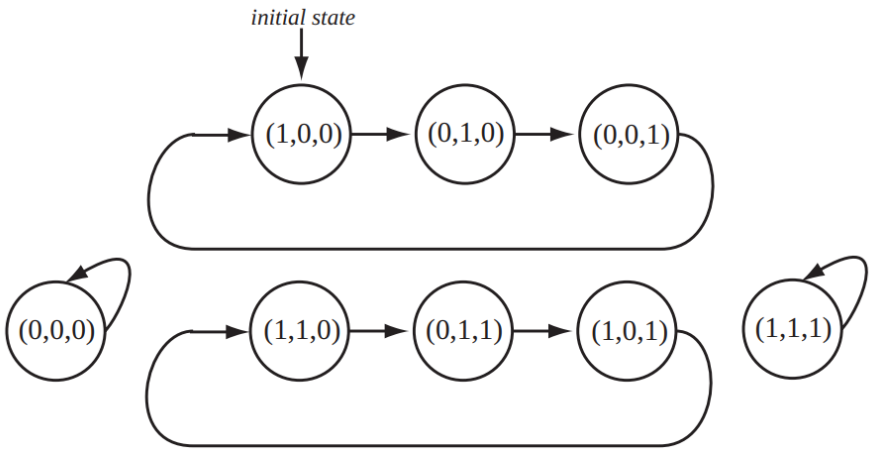


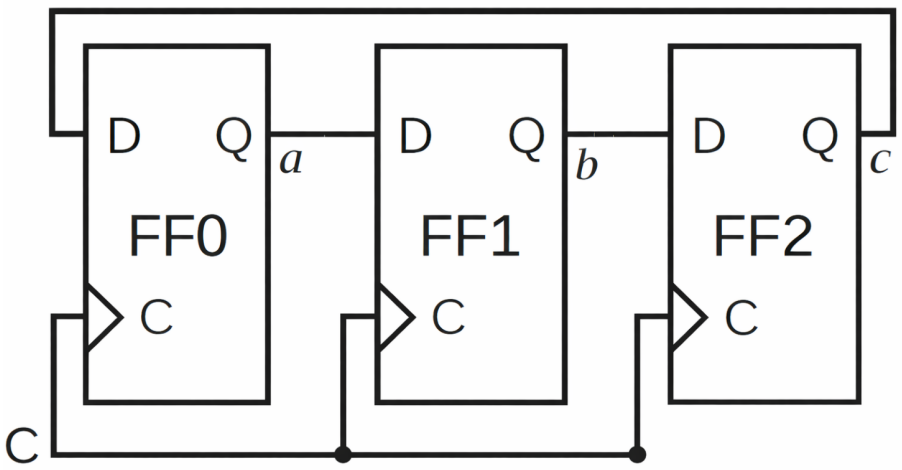
Figure: A 3-bit ring counter

State Transition System



Example

Hardware Circuit



State Transition System

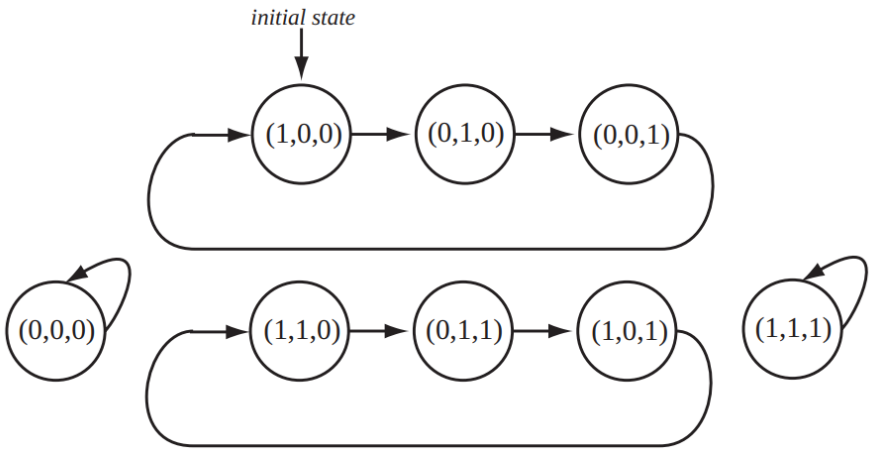
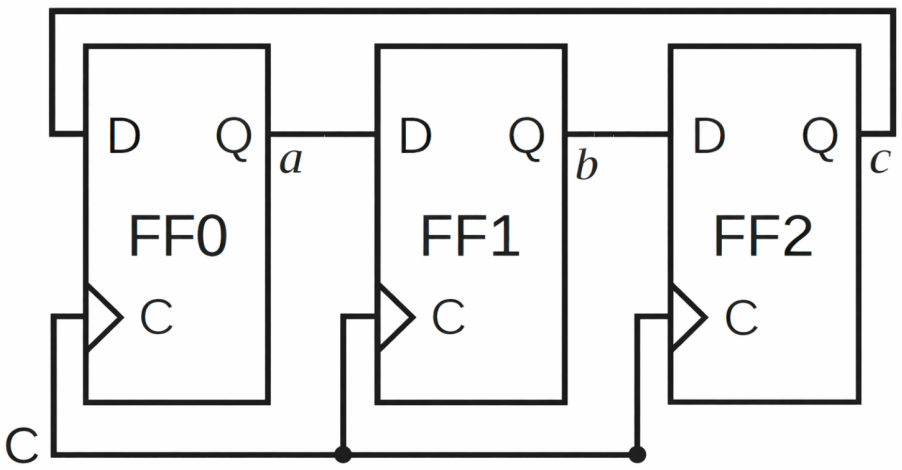


Figure: A 3-bit ring counter

- **Property 1:** Exactly one bit is high

Example

Hardware Circuit



State Transition System

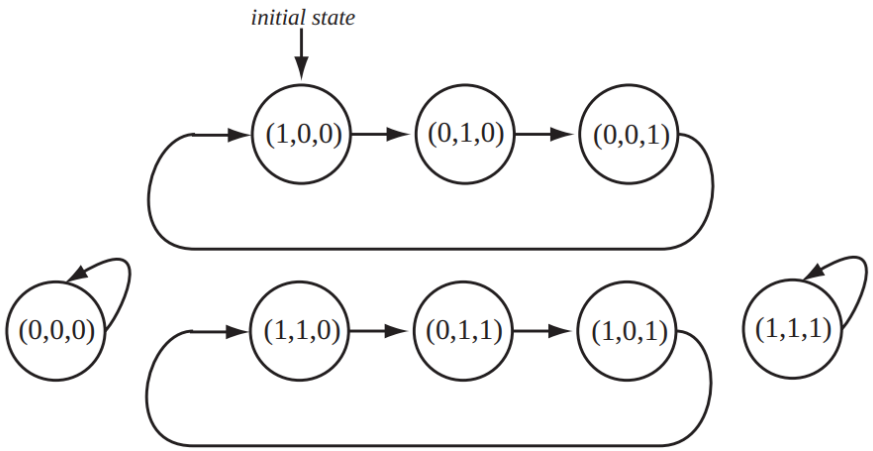


Figure: A 3-bit ring counter

- **Property 1:** Exactly one bit is high
- **Property 2:** $\neg c$ (signal c is never 1)

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Model Checking

Idea

Explore all reachable states of the system and check whether a property holds.

- System modeled as a finite state transition system
- Properties written in temporal logic
- Automatic bug finding
- Inventors received the **2007 Turing Award**

Main Challenge

State Explosion Problem

Handling State Explosion

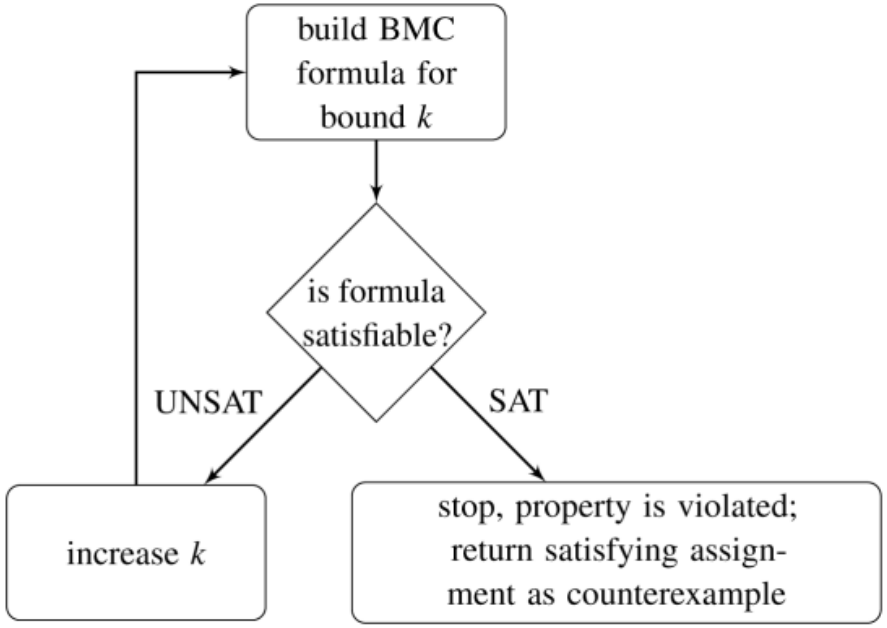
- Explicit state model checking (enumerate states)
- Symbolic model checking (BDDs)
- SAT-based model checking

SAT solving enabled scalable verification

Bounded Model Checking (BMC)

Key Idea

Search for a counterexample of length k using a SAT solver



SAT Encoding

$$I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \neg P(s_k)$$

BMC vs PDR (IC3)

Bounded Model Checking

- Searches bugs up to depth k
- Completeness only if bound sufficient
- Excellent bug finder
- Used in industry debugging

PDR / IC3

- Proves property for all time
- Builds inductive invariant
- No bound needed
- Completeness: safety proof

BMC finds bugs **PDR proves correctness**

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Related Work

Dynamic Property Ordering for PDR/IC3

- Multi-property verification sensitive to property order
- Uses runtime statistics from PDR proof engine
- Prioritizes easier properties first
- Improves convergence of unbounded verification

Limitation

Designed specifically for **inductive (IC3/PDR)** model checking

Motivation: Multi-Property Verification Challenge

Breadth-wise Multi-Property BMC (ABC)

- All properties advanced together
- Difficult property consumes solver effort
- Easy properties remain shallow

Easy properties are penalized

Questions

- Can properties be verified sequentially?
- Can ordering be decided dynamically?
- Can solver progress guide scheduling?

Goal: Improve depth exploration

Key Insight

A single difficult property can limit global exploration depth

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Our Contribution

- Dynamic ordering for multi-property **BMC**
- Iterative bounded solving
- Use runtime progress (frame depth reached)
- Reorder unsolved properties
- This work compares:
  - **ABC BMC**
  - **Dynamic-Ordering BMC (proposed)**

Goal

Improve bug-finding efficiency and convergence

Dynamic-Ordering BMC: Iterative Scheduling Algorithm

High-level Algorithm

1. **Iteration 0:**
   - Run BMC on each unsolved property for **1 second**.
2. **Iteration $i > 0$:**
   - Order all unsolved properties in **decreasing order of maximum frame** reached in iteration $i - 1$.
   - For each property in this order:
     - Run BMC with:

$$T_i = \begin{cases} T_{i-1}, & \text{if some properties were solved in iteration } i-1 \\ 2 \times T_{i-1}, & \text{otherwise} \end{cases}$$

3. Repeat until:
   - all properties are solved, or
   - the global time limit is reached.

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Experimental Setup

- **Benchmarks:** HWMCC'12 & HWMCC'13 (multi)
- **Time limit:** 3600 seconds
- **Architecture:** x86_64
- **CPU:** AMD Ryzen 5 5600X (12 threads)
- **Memory:** 128 GB RAM

Evaluation Metrics

- Number of SAT instances (#sat)
- Maximum frame reached

Experimental Results

Design	#In	#Out	#Lat	#And	Ckt Size	ABC BMC #SAT	ABC BMC Frame	Proposed BMC #SAT	Proposed BMC Frame
6s169.aig	58	16	145	2802	3021	0	10	0	1406
6s155.aig	201	32	513	13976	14722	0	1869	0	31564
6s292.aig	125	247	3190	13748	17310	1	669	1	1168909
6s154.aig	193	32	128	18183	18536	0	3	0	19168
6s284.aig	192	2	2352	19561	22107	0	97	0	97
6s327.aig	6285	33	3290	15906	25514	0	588	0	290
6s275.aig	844	673	3196	21571	26284	0	426	0	693
6s116.aig	4912	17	4922	22649	32500	16	66	16	59
6s340.aig	480	76	3337	42291	46184	0	23	0	37
bob12m05m.aig	437	10	3956	46101	50504	10	33	0	32
6s138.aig	439	16	4008	47702	52165	0	41	0	123
bob12m07m.aig	25	53	1258	63503	64839	0	18	0	22
6s175.aig	11684	3	7415	66693	85795	2	126	2	112
6s273.aig	983	42	15544	75718	92287	0	337	0	2440
6s381.aig	1210	932	12321	79513	93976	0	154	0	116814
6s409.aig	205	184	10523	120917	131829	0	94	0	81
6s406.aig	208	122	10746	123785	134861	0	49	0	125
6s404.aig	202	6	9801	126011	136020	1	62	1	39
6s407.aig	201	371	11379	129624	141575	0	93	0	105
6s386.aig	18612	13	14094	158198	190917	13	15	13	15
6s329.aig	1560	33	31947	169790	203330	0	32	0	44
6s401.aig	229	304	12309	218057	230899	0	10	252	58
6s137.aig	647	589	32922	299551	333709	0	14	0	36
6s387.aig	1121	409	29494	331746	362770	0	11	0	11
6s332.aig	83041	157	83717	1238400	1405315	0	11	0	90

Observations

- Proposed Dynamic-Ordering BMC reaches deeper bounds in several benchmarks
- Improvement observed in **64%** of cases
- No degradation in **76%** of benchmarks
- Particularly effective on medium-to-large circuits
- Baseline ABC BMC remains comparable on simple instances

Outline

1. Motivation
2. Verification Background
3. Model Checking and BMC
4. Related Work
5. Proposed Method
6. Experimental Results
7. Conclusion

Conclusion

- Proposed a **dynamic property ordering strategy** for multi-property BMC
- Uses runtime progress (frame depth) to guide scheduling
- Achieves deeper exploration in **64%** of benchmarks
- Causes no degradation in **76%** of cases compared to ABC BMC

Future Work

- Resource-optimized implementation
- Adaptive resource limit allocation across properties
- Ordering based on additional runtime statistics

References

E. Clarke, E. Emerson, A. Sistla,
Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications,
TOPLAS, 1986.

A. Biere et al.,
Bounded Model Checking,
Advances in Computers, 2003.

N. Eén, A. Mishchenko, R. Brayton,
Efficient Implementation of Property Directed Reachability,
FMCAD, 2011.

R. Brayton and A. Mishchenko,
ABC: An Academic Industrial-Strength Verification Tool,
CAV, 2010.

S. Das et al.,
PURSE: Property Ordering Using Runtime Statistics for Efficient Multi-Property Verification,
DATE, 2024.